E-COMMERCE RECOMMENDATION ENGINE — SYSTEM DESIGN V1

"Customers viewing this also viewed" — item-to-item at catalogue scale
Two-stage retrieval & ranking · sub-200 ms · cold start · feedback loops · evaluation

1 · REFERENCE ARCHITECTURE

online serving path above, offline training path below — they meet at the stores

ONLINE — synchronous, budget ≈ 150 ms p99, runs on every product-page view

Product page
viewing item P
user U (or anon)
session S

Recommendation API
context assembly
cache lookup
experiment assignment

STAGE 1 — CANDIDATE GENERATION · millions → ~500 · cheap, recall-oriented, run in parallel

- **Item-to-item CF** — co-view / co-purchase — precomputed top-N per item
- **Embedding kNN** — vector ANN on item P — semantic / visual similarity
- **Content-based** — category · brand · price band — works with zero interactions
- **Session sequence** — last N views this session — the only signal for anons
- **Complementary** — bought-together / basket — accessories, consumables
- **User history** — personalised, logged-in — two-tower retrieval
- **Popularity / trending** — category-scoped — the fallback that always works
- **Merchandised** — hand-curated slots — campaigns, new launches

Every source returns a scored list. Union them, dedupe by item id, keep the max score per source. Sources are independent — one failing must degrade the result, not the request. Each has its own timeout; whatever returned inside the budget goes forward.

STAGE 2 — RANKING
~500 → ~50 · precision

Learned ranker
features: item, user, context, pair (P → candidate), source, score, price delta, CTR priors
predicts P(click) / P(purchase)

Expensive per item — which is why stage 1 must cut first

STAGE 3 — POLICY
~50 → 8 shown

- drop out-of-stock
- drop the item being viewed
- drop already-purchased
- diversity: cap per brand/category
- price-band sanity vs P
- blocked / restricted items
- margin or campaign boost

Deterministic. Never a model.

RESPONSE
8 items + reason
+ impression id

"Because you viewed this item"

The impression id is what joins this response to the click that may follow — **without it you cannot measure anything.**

LATENCY BUDGET — the constraint that dictates the architecture

Stage	Budget (p99)	Why
Cache lookup	5 ms	hit ⇒ return immediately
Feature fetch	15 ms	online store, batched
Candidate generation	30 ms	parallel, each source capped
Ranking	40 ms	scales with candidate count
Policy + assembly	10 ms	pure computation
Total	**≈ 100 ms**	**50 ms headroom to 150**

Illustrative starting budget — measure against your catalogue and hardware, then set per-stage timeouts from the measurements.

SERVING STORES — read-only at request time; written only by the offline path

- **Item-to-item store (KV)** — item → top-N similar, scored — the single highest-value structure here
- **Vector index (ANN)** — item embeddings, HNSW / IVF — approximate by design — measure recall
- **Online feature store** — user, item and context features — must be point-in-time consistent with training
- **Item catalogue snapshot** — price, stock, status, attributes — drives the policy filters
- **Model registry** — versioned ranker artefacts — rollback is a config change

Response cache — keyed by (item, segment, experiment, model version)
Recommendations for a popular item are identical for most anonymous users. Cache aggressively; short TTL. Include the model version in the key or a deploy silently serves stale results from the previous model.

OFFLINE & STREAMING — asynchronous; nothing here is on the request path

writes the stores

Clickstream events — view · click · add-to-cart · purchase · impression — with impression id + position
Kafka — event backbone — partitioned by user
Stream processing — session features, counters, trending, recent views — seconds-fresh
Batch: co-occurrence — build item→item matrix, normalise, prune, top-N — hourly / daily
Batch: embeddings — train item vectors, index build → ANN — daily / weekly
Train ranker — labels from impressions + clicks + conversions, position-bias corrected
Offline evaluation — Recall@K · NDCG · coverage · diversity — gate before deploy
Deploy + A/B — shadow → % rollout, online metrics decide, rollback on guardrail breach

THE FEEDBACK LOOP — and why it is dangerous
The model decides what is shown. What is shown determines what is clicked. Clicks become tomorrow's training data. Left uncorrected, popular items get shown more, earn more clicks, and are ranked higher still — while the long tail becomes unreachable regardless of relevance. Correct with position-bias weighting, exploration, and a coverage guardrail.

THE SEPARATION THAT MAKES THIS WORK

Everything expensive — matrix building, embedding training, model fitting — happens offline and writes into a store. The request path only **reads**.

A recommendation request never trains, never scans the catalogue, and never joins across services. If it does any of those, the latency budget is already lost.

The consequence to accept: recommendations are **as fresh as the last pipeline run**. A product launched an hour ago is invisible to co-occurrence until it rebuilds — which is exactly why the content-based and streaming-trending sources exist alongside it.

2 · ITEM-TO-ITEM COLLABORATIVE FILTERING

the workhorse for "customers who viewed this also viewed"

Why item-to-item, not user-to-user

The classic alternative finds users similar to you, then recommends what they liked. It does not scale: user vectors change constantly and there are more users than items. **Item-to-item inverts this** — compute item similarity offline, look it up online. Item pairs are far more stable than user preferences, so the expensive part is computed rarely and the online step is a key lookup. This is the approach Amazon published in 2003 and it remains the baseline every newer method is measured against.

The computation

```
# 1 · co-occurrence from session or order data
count(i,j) = sessions where i and j both appear

# 2 · normalise — raw counts just rank popularity
sim(i,j) = count(i,j) / sqrt(count(i) × count(j))
           cosine on binary interaction vectors

# 3 · prune, then keep top-N
drop pairs with count(i,j) < min_support
store item → [(j, sim) … top 100]
```

Step 2 is the one that gets skipped, and skipping it breaks everything. Raw co-occurrence recommends the best-selling item on every page, because it co-occurs with everything. Normalising by each item's own frequency is what converts "popular" into "actually related".

Choosing the signal

Signal	Character
Co-view	Dense, fast to accumulate, noisy. Finds **substitutes** — alternatives to what you are looking at
Co-purchase	Sparse, high intent. Finds **complements** — what goes with it
View→purchase	"Viewed this, bought that" — strong signal for the conversion goal
Same-basket	Explicit complements; best for accessories and consumables

Substitutes and complements are different products and belong in different widgets. On a camera page, co-view gives other cameras; co-purchase gives lenses, bags and cards. Mixing them into one list produces a widget that does neither job. Compute both, label the slot.

Practical parameters — tune, don't copy

Knob	Effect
min_support	Too low: noise from coincidence. Too high: long tail loses all recommendations
Session window	What counts as "together" — a session, a day, an order
Top-N stored	Storage vs downstream room to re-rank; 50–200 typical
Rebuild cadence	Freshness vs compute cost
Decay	Weight recent interactions higher so seasonality is tracked

Scale note. The full item×item matrix is quadratic and unnecessary. Build it from inverted session lists, prune aggressively, and keep only top-N per item. Most catalogues reduce to a few hundred megabytes — small enough to serve from a plain key-value store.

3 · EMBEDDINGS & ANN

where co-occurrence runs out

Co-occurrence can only relate items that have **already been seen together**. A new product, or one in the long tail, has no pairs — so it is invisible. Embeddings solve exactly that: they place every item in a vector space where proximity means similarity, whether or not anyone has co-viewed them.

Where the vectors come from

Source	Captures
Behavioural — sequence models over sessions	Learned relatedness — items viewed in similar contexts land close together
Text — title, description, attributes	Semantic similarity; **works on day one, no interactions needed**
Visual — product imagery	Look-alike — decisive in fashion, furniture, décor
Hybrid	Concatenate or learn a joint space; usually the strongest

Serving

```
query = embedding(item P)
ANN.search(query, k=200, filter=tenant/catalogue)
   → 200 nearest items in ~5–15 ms
```

Filter before you search, not after. Post-filtering an ANN result for stock, category or price can leave you with far fewer than k items — sometimes zero. Push the filter into the index so the k returned are already valid.

ANN is approximate by construction. A genuinely similar item can simply not be returned. Measure recall against exact search on a sample before assuming the retrieval layer is exhaustive — the tuning knob is the search-effort parameter, and it trades recall against latency.

Embeddings go stale silently. Nothing errors when the index is a week old and the catalogue has moved — results just quietly get worse. Track index age and item coverage as first-class metrics, and alert on both.

4 · COLD START

three different problems

Case	Problem	Approach
New item	No interactions, so no co-occurrence pairs exist	Content and text embeddings from metadata; borrow from the nearest catalogue neighbours; give it forced exposure to bootstrap signal
New / anonymous user	No history to personalise from — the majority of storefront traffic	**Session-based**: the current item plus the last few views is enough. This is the default path, not the exception
New catalogue — a new merchant	No behavioural data at all	Content similarity only; category popularity priors; consider a shared cross-tenant model only where privacy permits

Design for anonymous first. A large share of product-page views are from users with no profile. If the design assumes a rich user history, the common case falls through to a popularity fallback and the feature underperforms while looking fine in offline evaluation on logged-in users.

The exposure problem

A new item cannot earn interactions until it is shown, and it will not be shown until it has interactions. Breaking that loop needs deliberate **exploration**: reserve a slot, or add a small randomised component, so new items get impressions they have not yet earned.

Exploration costs measurable revenue in the short term and pays back through catalogue coverage. Budget it explicitly — a percentage of slots — rather than leaving it implicit, so the cost is visible and tunable rather than argued about.

Fallback chain

```
item-to-item      → if empty ↓
embedding kNN     → if empty ↓
same category top → if empty ↓
category trending → if empty ↓
store bestsellers   ← always returns
```

The widget must never render empty. An empty recommendation slot on a product page is a visible defect. The last link in the chain must be something that always returns.

5 · RANKING & TRAINING DATA

where most of the quality — and most of the subtle error — lives

Feature families

Family	Examples
Item	Price, rating, review count, age, category, brand, historical CTR and conversion
Pair (P → c)	Co-occurrence score, embedding distance, same brand, price ratio, category relation
User	Purchase history, category affinity, price sensitivity, recency
Context	Device, time of day, referrer, session depth
Source	Which retriever produced it, and its rank within that source

The pair features carry most of the signal. "How related is this candidate to the item being viewed" is the actual question on a product page — item-level popularity features will otherwise dominate and quietly turn the ranker into a bestseller list.

Objective

Optimise for	Consequence
Click	Abundant labels; drifts toward clickbait and cheap items
Add-to-cart	Intermediate intent; a reasonable proxy
Purchase	Sparse labels; aligned with the business
Weighted blend	Usually the practical answer — weights are a business decision, not a modelling one

Position bias — the defect that silently poisons training

Items in slot 1 get clicked far more than slot 8 **regardless of relevance**, simply because they are seen first. Train naively on click logs and the model learns "things we ranked highly get clicked" — a self-confirming loop that mistakes its own past decisions for user preference.

You must log the position and correct for it. Weight each impression inversely to its position's examination probability, or estimate that probability from randomisation. A ranker trained on uncorrected click logs will look excellent offline and fail to improve anything online — the most common way a recommender project produces no measurable lift.

What to log per impression

```
{ impression_id, session_id, user_id|null,
  anchor_item: "P",
  shown: [{item, position, source, score}…],
  model_version, experiment_arm,
  timestamp }

// then, joined later by impression_id:
click / add_to_cart / purchase events
```

Log what was shown, not only what was clicked. Negatives are the items displayed and ignored. Without the impression record there are no true negatives, and the training set is unusable no matter how much click data accumulates.

Log the model version and experiment arm too. Without them you cannot attribute a metric movement to a change, and every A/B result becomes unfalsifiable.

6 · EVALUATION

offline gates, online decides

Offline — a gate, never a verdict

Metric	Answers
Recall@K	Did the candidate set contain the item the user actually engaged with?
NDCG@K	Was it ranked near the top?
Catalogue coverage	What share of the catalogue is ever recommended?
Intra-list diversity	Are the eight shown meaningfully different from each other?
Novelty	Are we surfacing anything the user would not have found alone?

Offline metrics are computed on logged data the previous model produced, so they systematically favour models that behave like the incumbent. Treat them as a gate that blocks obvious regressions — never as evidence that a change is an improvement.

Online — the only real evidence

Metric	Watch for
Widget CTR	Easy to move by showing cheap items; never read alone
Attributed conversion	The metric that matters
Revenue per session	Catches cannibalisation that CTR hides
AOV impact	Complements should raise basket size

Guardrails — abort regardless of lift

- **p99 latency** within budget
- **Coverage** not collapsing onto a few thousand items
- **Empty-widget rate** near zero
- **Out-of-stock rate** in shown results at zero

A variant that lifts CTR while halving catalogue coverage is not a win. It has concentrated demand onto the head of the catalogue, and the damage compounds through the feedback loop long after the experiment concludes.

7 · FAILURE MODES

most are silent — the widget renders, the results are just quietly wrong

Failure	Symptom & fix
Raw co-occurrence, unnormalised	Bestsellers on every page. Normalise by item frequency
Popularity feedback loop	Coverage shrinks each retrain. Position-bias correction + exploration + coverage guardrail
Position bias untreated	Great offline metrics, no online lift. Log position, weight accordingly
Out-of-stock recommended	Click leads to a dead end. Filter in stage 3 against a fresh catalogue snapshot
Self-recommendation	The viewed item appears in its own widget. Explicit exclusion
Stale index	Quality degrades with no error. Alert on index age and coverage

Failure	Symptom & fix
No diversity cap	Eight near-identical variants of one product. Cap per brand and per category
Substitutes in a complements slot	Cameras beside a camera instead of lenses. Separate the models, label the slot
Cache keyed without model version	Deploy changes nothing until TTL expiry. Include version in the key
Training / serving feature skew	Offline good, online bad. Compute features once, share the definition
Recommending just-purchased items	Another washing machine. Filter recent purchases for durables
No impression logging	Cannot train, cannot measure. Log shown-and-ignored from day one

The unifying property: none of these throw an error. The page renders, the widget fills, latency is fine, dashboards are green — and the recommendations are worse than random in a way only a metric or a human eye will catch. Build the metrics before the model.

8 · BUILD ORDER

what to ship first, and why the order matters

Do not start with a neural ranker. Normalised item-to-item co-occurrence plus a popularity fallback captures a large share of the achievable value for a fraction of the effort — and it becomes the baseline every later model must beat. Without that baseline you cannot tell whether the sophisticated version is helping.

Phase	Ship
0	**Instrumentation.** Impression logging with position, model version and experiment arm. Nothing else works without it
1	Normalised co-view item-to-item + popularity fallback. Cache. Measure CTR and conversion
2	Policy layer: stock, self, purchased, diversity cap
3	Co-purchase model for a separate complements slot
4	Content and text embeddings + ANN for cold start and the long tail
5	Learned ranker over the union, position-bias corrected
6	Personalisation from user history; session sequence models
7	Exploration budget and coverage guardrails

Phase 0 is not optional and cannot be retrofitted cheaply. Every later phase trains on data logged in phase 0. Start logging impressions before the first model ships, even while the widget is still showing bestsellers — that data becomes the training set.

A note on scale. Everything here assumes a catalogue where the item-to-item structure fits comfortably in a key-value store and the ANN index fits in memory. Most SMB and mid-market catalogues do. Re-derive the numbers before assuming they hold at a different order of magnitude.

Ship phases 1 and 2 together. A working item-to-item model without the policy layer will recommend out-of-stock items and the product being viewed — which reads as broken to a customer regardless of how good the underlying relevance is.

9 · PRODUCTION READINESS CHECKLIST

confirm before the widget goes on the product page

- ☐ **Impressions logged** with item, position, source, score
- ☐ **Model version and experiment arm** on every impression
- ☐ **Impression id joins** to click, cart and purchase events
- ☐ **Co-occurrence normalised**, not raw counts
- ☐ **min_support tuned** against long-tail coverage
- ☐ **Substitutes and complements** served in separate slots
- ☐ **Every retrieval source has a timeout** and degrades independently
- ☐ **Fallback chain terminates** in something that always returns
- ☐ **Widget never renders empty**
- ☐ **Viewed item excluded** from its own results
- ☐ **Out-of-stock filtered** against a fresh catalogue snapshot
- ☐ **Recently purchased filtered** for durables
- ☐ **Diversity cap** per brand and per category
- ☐ **ANN recall measured** against exact search on a sample
- ☐ **ANN filters applied pre-search**, not post
- ☐ **Index age and item coverage** monitored and alerted
- ☐ **Cache key includes model version**
- ☐ **Position bias corrected** in training
- ☐ **Feature definitions shared** between training and serving
- ☐ **p99 latency within budget** under peak load
- ☐ **Per-stage timeouts** derived from measurement
- ☐ **Offline gate** blocks regressions before deploy
- ☐ **A/B framework** with guardrail auto-abort
- ☐ **Catalogue coverage tracked** as a first-class metric
- ☐ **Exploration budget** explicit and tunable
- ☐ **Rollback** is a config change, not a redeploy

The test that summarises the sheet. Take a product with exactly one historical co-view, a brand-new product with none, and your best-selling product. Ask for recommendations on all three. If any returns an empty widget, an out-of-stock item, itself, or eight near-identical variants, the system is not ready — **and none of those cases will raise an error to tell you.**